RescueBench Agent: A ReAct-Based Emergency Response Allocation Agent with Deterministic Safety Validation

Akshat Bhat Trusha Talati Akhil Gogineni Nicolas Yepes
Department of Computer Science, University of Illinois Urbana-Champaign
Champaign, IL 61820, USA
{akshatb5, ttalati2, vg23, nyepes2}@illinois.edu

1 Introduction

Emergency response dispatch is a domain where planning failures carry severe real-world consequences. Routing an ambulance through a blocked road, dispatching a vehicle lacking the required capabilities, or failing to transport patients to a hospital before a deadline are physically impossible or harmful actions. Despite the growing use of LLM-based agents for planning and decision-making, existing systems lack mechanisms to prevent such hallucinated but plausible actions from being executed.

The central challenge we address is the inability of LLMs to reliably route vehicles and respond to requests that minimize cost and increases efficiency. While LLMs excel at parsing unstructured semantic data, they frequently fail at strict spatial reasoning and multi-step constraint satisfaction(Song, 2026). Current LLM solvers may hallucinate mathematical values, lose track of physical vehicles, and return routes that are physically or practically impossible.

Our agent addresses this gap with three design contributions:
(1) **WorldState** simulates a world where the agent will run. It contains the map of a city and resource states
(2) **ValidatorTool** Will validate each action taken by agent and verify that it is a possible action given the current world state
(3) **ReAct orchestration loop**:r The model reasons over tool outputs and iteratively refines its plan to allocate resources and create a solution. A greedy deterministic planner serves as a fallback when the LLM is unavailable or budget-constrained.

Preliminary evaluation on the Tier 1 "Basic Triage" scenario demonstrates complete resolution of all incidents within deadlines at zero constraint violations in both deterministic and LLM-guided modes.

2 Related Work

ReAct-style agents. (Yao, 2023) introduced the ReAct framework, which interleaves chain-of-thought reasoning with tool-use actions. Our agent adopts this paradigm directly, using Claude's native function-calling interface as the action mechanism.

LLMs for vehicle routing. (Zhang, 2025) propose an agentic framework for solving complex vehicle routing problems with LLMs, finding that models perform best as heuristic orchestrators over exact solvers rather than as end-to-end mathematical solvers. Our architecture validates this: the LLM selects which vehicle to dispatch and why, while Dijkstra's algorithm handles path computation. Additionally, (Li et al., 2025) showed that LLMs often produce infeasible routes under certain constraints which is why we designed the `ValidatorTool`.

Constraint enforcement in agent systems. (Wang et al., 2025) identify constraint satisfaction decreases LLM based decision making. Some architectures rely on the LLM to self-enforce constraints through prompting. Our approach externalizes constraint enforcement entirely to a deterministic python layer to separate concerns and force LLM to follow constraints.

3 Agent Design

3.1 Architecture Overview

The agent consists of four tightly coupled components. Figure 2 shows the information flow. **WorldState** is the single source of truth for the simulation, holding the city graph (nodes and edges), blocked edges, vehicle status, incident status, and the simulation clock. It exposes a `dijkstra()` method for shortest-path queries and a `build_graph()` method that dynamically excludes blocked edges, ensuring path computations always reflect the current passable edges.

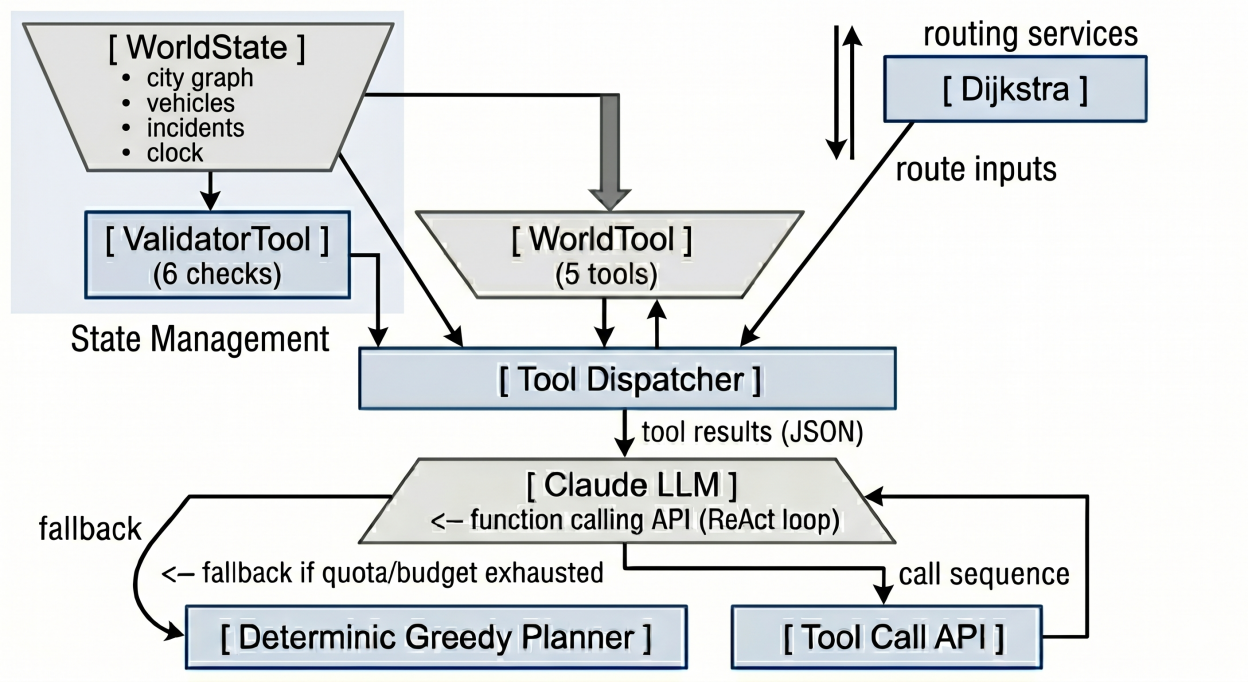

Figure 1: Scenario Processing System Architecture

Figure 2: Agent architecture. All state mutations flow through the ValidatorTool before reaching WorldState.

WorldTool is the agent's read/write interface to WorldState. It exposes five callable tools: `get_map_state`, `get_vehicles`, `get_incidents`, `get_shortest_path`, and `dispatch_vehicle`. The only way for LLM to modify world state is through `dispatch_vehicle`.

ValidatorTool is a Python safety layer positioned that validates every dispatch against six physical constraint categories: vehicle and incident existence, vehicle availability, incident resolution status, capability matching, patient capacity, route passability, and hospital existence and capacity. Failures increment a `violation_count` and an error message to the LLM for self-correction.

Agent implements the ReAct orchestration loop. It sends the initial scenario context to the LLM, processes tool-call responses in each turn, routes calls through the Validator and WorldTool, and feeds results back to the LLM.

3.2 Technical Approach

Input processing. At the start of each run, the agent calls `get_map_state` and `get_incidents` before reasoning about any dispatch ensuring the model grounds its plan in actual current topology rather than hallucinated state.

Planning and reasoning. The LLM reasons in ReAct style: after calling the `dispatch_vehicle` the agent will go through the verification python layer to make sure decision satisfies constraints. Upon a validation error, analyze the error and never repeat the same invalid action. if no validation errors the system prompt encodes four priority

heuristics: (1) always verify routes before dispatching; (2) prioritize higher-severity incidents while accounting for deadline urgency; (3) always specify a `hospital_node` when dispatching ambulances with patients;

Execution. When the LLM issues a `dispatch_vehicle` call, execution follows a strict sequence:
1. `validate_dispatch()` checks all physical constraints against the live WorldState.
2. If validation fails, a structured error is returned as the tool result and no world state is mutated.
3. If validation passes, `WorldTool.dispatch_vehicle()` executes the dispatch, runs Dijkstra to compute the actual route and updates vehicle availability. For ambulance dispatches involving patients, the method also computes the route to the specified hospital and returns total trip time. This creates a closed self-correction loop

Memory and state. The agent is stateless across LLM calls—all context is passed explicitly through the Claude chat history. WorldState serves as the persistent physical memory (vehicle positions, incident resolution status, hospital occupancy) and is updated synchronously with each successful dispatch. The LLM cannot directly access or mutate WorldState; all interactions are mediated through the tool interface and validation mechanisms.

3.3 Implementation Details

Core Setup. The agent uses `claude-sonnet-4-5` with temperature fixed at 0.0. We leverage Claude's native function-calling API, which returns structured, typed arguments. This eliminates the need to parse tool calls from free-form text—a major source of fragility in standard text-based ReAct implementations.

Error Handling & Fallback. The agent relies on a three-layered error architecture:
- *Constraint Errors:* Returned as `VALIDATOR ERROR: <reason>`, explicitly prompting the model to self-correct.
- *API & Rate Limits:* Triggers an exponential-delay retry loop parsed directly from the error message.
- *Budget Exhaustion:* Activates the deterministic planner (`ALLOW_DETERMINISTIC_FALLBACK=1`). It ranks incidents by severity and deadline, then dispatches the nearest capable vehicle. It guarantees validity by routing all actions through the `ValidatorTool`.

3.4 Design Decisions

- *Decoupled Validation:* We rejected embedding validation inside `dispatch_vehicle` to avoid conflating state mutation with constraint checking. This ensures validation can be tested in isolation without risking partial state mutation.
- *Deterministic Fallback:* Ensures benchmark completability under strict API limits and provides a hard baseline to measure the LLM's marginal reasoning contribution.
- *Native Function Calling:* Chosen over prompt-based tool use strictly to bypass the fragility of free-text parsing.

4 Experimental Setup

Benchmark. We evaluate our approach on a dataset of 20 deterministic JSON environments modeling a 16-node city graph with a heterogeneous 4-class vehicle fleet across four difficulty tiers. Our primary evaluation focuses on the *Tier 1* scenario. This scenario challenges the system with two concurrent incidents: a minor medical emergency and a small structural fire. This requires single-unit dispatches on an unobstructed graph.

Baselines. We compare against two baselines:
- *Greedy Heuristic Solver* (deterministic fallback): Ranks incidents by severity and deadline, then dispatches the nearest capable vehicle using Dijkstra's algorithm.
- *Zero-Shot LLM* (no validator): The model operates with identical prompts and tools but lacks the `ValidatorTool`. Dispatches execute directly without constraint checking, isolating the impact of the safety validation layer.

Evaluation Metrics. Following the RescueBench protocol, we report:
- Priority-Weighted Resolution Score (PWRS): $\sum(w_i \cdot I_i)$ normalized by total severity weight.
- Resolution Rate and *Deadline Adherence Rate*.
- Constraint Violation Count and Step Efficiency (resolved incidents per reasoning step).

Experimental Details. Each scenario is run 3 times to satisfy the benchmark's aggregation requirement. The maximum number of LLM calls per run is strictly capped at 30. If this budget is exhausted, the deterministic fallback activates (controlled via `ALLOW_DETERMINISTIC_FALLBACK=1`).

5 Results

5.1 Overall Performance

Table 1 shows preliminary single-run results on Tier 1 Basic Triage.

Table 1: Preliminary results on Tier 1 Basic Triage (single-run).

Agent Mode	PWRS	Res. Rate	Deadline Adh.	Violations	Step Eff.
Deterministic (Greedy)	1.000	1.00	1.00	0	≈ 0.67
LLM-Guided (Claude)	1.000	1.00	1.00	0	≈ 0.18
Zero-Shot	pending	–	–	–	–

Both agents achieve perfect completion with no violations. The LLM-guided agent is less step-efficient due to additional tool calls.

5.2 Performance by Task Type

Deterministic. Resolves both incidents in 2 dispatches with no wasted actions. Prioritizes the fire (severity 6) over medical (severity 4). Arrival times: 11 min (fire) and 18 min (medical), both within deadlines. No interpretability.

LLM-Guided. Uses 7 steps (state retrieval, path planning, dispatch, verification). Correctly matches vehicles to capabilities and selects valid routes and hospital. Achieves full resolution with no violations but higher overhead.

5.3 Qualitative Analysis

Success. The LLM plans paths (11 min to fire, 18 min to medical) and completes both tasks within deadlines, verifying via `report_status`.

Failure + Recovery. In earlier runs, the LLM assigned an ambulance to a fire. The validator rejected this, and the agent corrected to FIRE_01, demonstrating effective self-correction with minor efficiency cost.

Takeaway. Deterministic is efficient but opaque; LLM-guided is interpretable and robust but less efficient.

6 Discussion

What works. Constraint enforcement via `ValidatorTool` proves highly effective. In all LLM-guided runs observed, violations were caught at the validator layer and never propagated to world state. The LLM consistently revised plans after receiving validator errors, with no repeated invalid actions observed. Moreover, separation of routing computation from LLM reasoning also prevents route hallucination: because the LLM only specifies source and destination for `get_shortest_path` rather than specifying the path itself, it cannot hallucinate a route through a blocked edge.

What doesn't work. LLM step efficiency is significantly lower than the deterministic planner—the LLM uses 4–6x more steps per resolution due to intermediate verification calls. Zero-Shot mode without the validator is expected to produce constraint violations (particularly capability mismatches and patient capacity errors) based on patterns observed in preliminary testing; formal quantification is pending.

Limitations. The current agent does not implement lookahead or multi-step planning—it reasons greedily over the current incident set without anticipating future vehicle availability or cascading deadline conflicts. On Tier 3 forced-scarcity scenarios, this may produce suboptimal priority allocation. The agent also does not exploit parallel dispatching: on scenarios with multiple simultaneous incidents, the LLM dispatches sequentially rather than issuing parallel tool calls.

7 Conclusion

We presented a ReAct-based emergency response allocation agent whose core contribution is a clean architectural separation between LLM-driven semantic reasoning and deterministic physical constraint enforcement. The `ValidatorTool` prevents hallucinated-but-impossible actions from mutating world state, while structured error messages enable reliable LLM self-correction. Moreover, a greedy Dijkstra-based fallback ensures benchmark completability under API constraints. Our current results confirm complete incident resolution with zero constraint violations in both planning modes. Future work will be evaluating the agent on alternative scenarios, where forced scarcity and mid-mission graph disruptions present the harder planning challenges. We will also integrate a mechanism for the model to output its reasoning behind its decisions to make it's thought process easier to track.

References

[Bhat et al.(2026)] A. Bhat, T. Talati, A. Gogineni, and N. Yepes. RescueBench: Evaluating large language models on dynamic spatial reasoning and constraint satisfaction. *Draft Benchmark Paper*, University of Illinois Urbana-Champaign, 2026.

[Li et al.(2025)] K. Li et al. Multi-objective infeasibility diagnosis for routing problems using large language models. *arXiv preprint arXiv:2508.03406*, 2025.

[Song et al.(2026)] P. Song et al. Large language model reasoning failures. *arXiv preprint arXiv:2602.06176*, 2026.

[Wang et al.(2025)] Y. Wang et al. Large language models in operations research: Methods, applications, and challenges. *arXiv preprint arXiv:2509.18180*, 2025.

[Yao et al.(2023)] S. Yao et al. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations*, 2023.

[Zhang et al.(2025)] N. Zhang et al. An agentic framework with LLMs for solving complex vehicle routing problems. *arXiv preprint arXiv:2510.16701*, 2025.